

Handout: 725103 Data Structure and Algorithm

สัปดาห์ที่ 1: ภาพรวมรายวิชา • Data Structures • Data in Memory • ADT

ภาคปลาย ปีการศึกษา 2568 | อาจารย์ผู้สอน: ปพนธ์ร์ แยมโซติ

เป้าหมายการเรียนรู้ประจำสัปดาห์ (Learning Outcomes)

เมื่อจบคาบนี้ นักศึกษาสามารถ...

- อธิบายความหมายของ *data* / *data structure* / *algorithm* และความสัมพันธ์ระหว่างกันได้
- อธิบายว่า *Data Structures* ช่วยเรื่อง *efficiency* (เวลา/ความสะดวก/ทรัพยากร) อย่างไรได้
- อธิบายแนวคิดพื้นฐานของ *memory* แบบภาพ และความหมายของ *address* ได้
- อธิบายแนวคิด *ADT (Abstract Data Type)* และยกตัวอย่าง *operation* ของ ADT อย่างน้อย 2 แบบได้
- ทำกิจกรรม *Think-Pair-Share* และให้เหตุผลเชิงแนวคิดได้อย่างเป็นระบบ

1 คำศัพท์สำคัญ (Key Terms)

คำศัพท์	ความหมายแบบสั้น	ตัวอย่าง/หมายเหตุ
<i>data</i>	ข้อมูลดิบ/ข้อเท็จจริงที่ยังไม่ถูกจัดระเบียบ	เช่น รายชื่อ, คะแนน, เวลา, พิกัด
<i>data structure</i>	วิธี “จัดระเบียบข้อมูล” เพื่อให้ทำงานกับข้อมูลได้มีประสิทธิภาพ	เช่น <i>array</i> , <i>linked list</i> , <i>stack</i> , <i>queue</i> , <i>tree</i> , <i>graph</i>
<i>algorithm</i>	ขั้นตอนการทำงาน (step-by-step) เพื่อแก้ปัญหา	เช่น วิธีค้นหา/เรียงลำดับ
<i>efficiency</i>	ความคุ้มค่าในการใช้เวลา/ทรัพยากร	สัปดาห์นี้วัดเชิงแนวคิด; สัปดาห์ถัดไปจะเข้าสู่ Big-O
<i>memory</i>	พื้นที่เก็บข้อมูลระหว่างโปรแกรมทำงาน (คิดเป็นบล็อกล)	แต่ละบล็อกลมี <i>address</i>
<i>address / pointer</i>	หมายเลขอ้างอิงตำแหน่งใน <i>memory</i>	เปรียบเทียบ “เลขที่บ้าน” ของข้อมูล
<i>ADT (Abstract Data Type)</i>	นิยามชนิดข้อมูลเชิงนามธรรม: ระบุว่า “ทำอะไรได้บ้าง” ไม่ผู้ว่า “ทำอย่างไร”	เช่น <i>Stack ADT: push/pop/top</i>
<i>operation</i>	ความสามารถ/คำสั่งที่ ADT รองรับ	เช่น <i>insert</i> , <i>delete</i> , <i>search</i> , <i>isEmpty</i>
<i>trade-off</i>	ได้อย่างเสียอย่าง: เลือกให้เหมาะกับโจทย์	เร็วขึ้นแต่อาจกินหน่วยความจำมากขึ้น

Your Note: please feel free to write notes anywhere you want.

2 ทำไมต้อง Data Structures และ Algorithms?

แนวคิดสำคัญ: เรามี “ข้อมูล” เยอะขึ้นเรื่อย ๆ และต้องทำงานซ้ำ ๆ (ค้นหา/เพิ่ม/ลบ/จัดเรียง) ถ้าเก็บข้อมูลแบบไม่เป็นระบบ เราจะเสียเวลาและมีโอกาสผิดพลาดมากขึ้น

แบบฝึกหัดที่ 1 (ในคาบ): โต๊ะรก vs โต๊ะเป็นระเบียบ

- โต๊ะรก: ของกระจุกกระจาย ต้องรื้อหา → ใช้เวลามาก
- โต๊ะเป็นระเบียบ: แบ่งหมวด/มีช่อง/มีป้ายกำกับ → หาของเร็วขึ้น
- *Data structure* คือ “แบบแผนการจัดของ” ส่วน *algorithm* คือ “วิธีเดินไปหยิบของ”



สถานการณ์: คุณต้องหาเอกสาร “ใบเสร็จหมายเลข 017” จากกองเอกสาร 30 แผ่น

1. ถ้าเอกสาร “กองรวม” ไม่มีการจัดหมวด (โต๊ะรก) คุณคิดว่าโดยเฉลี่ยต้องเปิดดูประมาณกี่แผ่นถึงจะเจอ? อธิบายเหตุผล (ถ้าสามารถวิเคราะห์เชิงความน่าจะเป็นได้จะดีมาก)
2. ถ้าเอกสารถูกจัดเป็นแฟ้ม 3 หมวด (ใบเสร็จ/ใบกำกับ/ใบเสนอราคา) และใบเสร็จเรียงตามหมายเลข คุณคิดว่าจำนวนขั้นตอนลดลงอย่างไร? (ยังไม่ต้องใช้ Big-O)

3. ถ้าตัดเรื่องความเป็นระเบียบ ความสะอาด สนใจแค่เรื่องเวลาที่ใช้กับสิ่งของบนโต๊ะ ทั้งสองแบบมีข้อดีข้อเสียต่างกันอย่างไร หรือมี use case อะไรบ้างในแต่ละแบบ



4. สรุป *trade-off* ของการจัดแฟ้ม (ต้องเสียอะไรเพื่อให้หาง่ายขึ้น?)

Think–Pair–Share (TPS)

คำถาม: “ทำไมการจัดเก็บข้อมูลให้เป็นระบบ จึงช่วยให้การทำงานเร็วขึ้นได้ แม้ข้อมูลจะมีจำนวนมากขึ้น?”

1. **Think:**

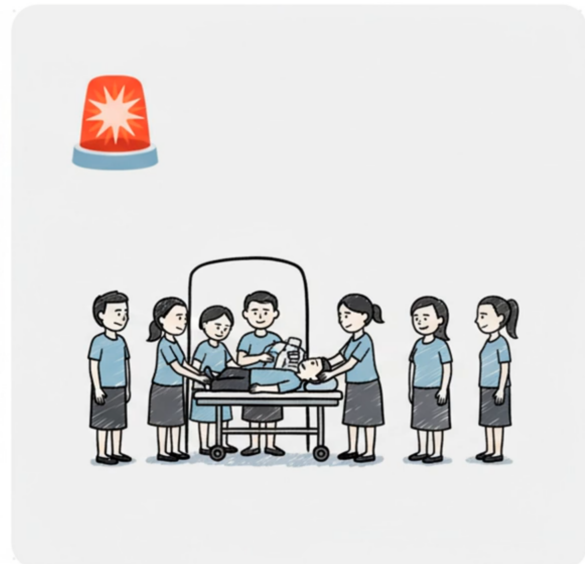
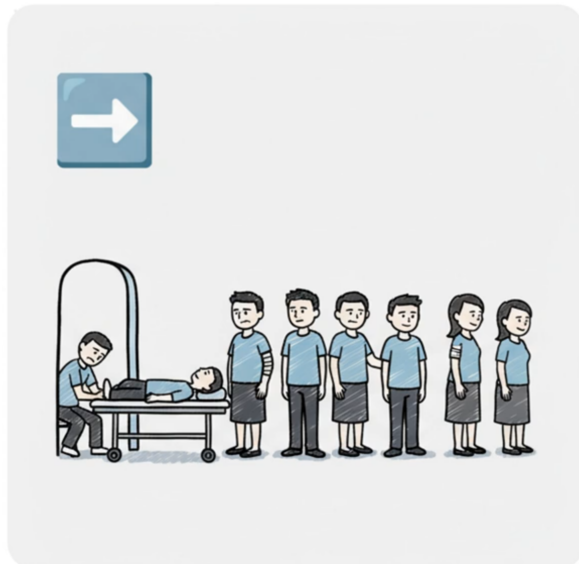
2. **Pair:** แลกเปลี่ยนกับเพื่อน แล้วสรุปประเด็นร่วมกัน

3. **Share:** เขียนประโยคสรุปของกลุ่ม

แบบฝึกหัดที่ 2: คิวเข้ารับบริการ (มองเชิงระบบ: ต้องเก็บอะไร? ต้องทำอะไร?)

เป้าหมาย: ให้นักศึกษามองเห็นว่า “การตัดสินใจว่าใครได้พบแพทย์ก่อน” ต้องอาศัย ข้อมูลที่ต้องเก็บ และ กระบวนการทำงาน ที่ต่างกันในแต่ละสถานการณ์

1. เข้าคิวรับการรักษายาบาลใน OPD ในเวลา
2. เข้าคิวรับการรักษายาบาลในแผนกฉุกเฉิน



A) ระบุ “ข้อมูล” ที่จำเป็นต้องเก็บ (Data needed) ให้เติมตารางด้านล่าง โดยตอบว่า “ข้อมูลนี้จำเป็นไหม” สำหรับ OPD และ ER พร้อมเหตุผลสั้น ๆ (สนใจแค่เรื่องลำดับการเข้ารับการรักษา ไม่สนใจเรื่องการวินิจฉัยที่เป็นหน้าที่ของแพทย์)

ข้อมูลที่ต้องเก็บ	OPD	ER	เหตุผล (สั้น ๆ)
เวลามาถึง/ลำดับการมาถึง			
นัดหมาย/Walk-in			
ความรุนแรงของอาการ			
อาการสำคัญ (เช่น แน่นหน้าอก/หายใจลำบาก)			
อายุ/กลุ่มเสี่ยง			
สถานะปัจจุบัน (รอ/กำลังตรวจ/เสร็จแล้ว)			
อื่น ๆ ที่กลุ่มคิดว่า “ขาดไม่ได้”			

สรุป: OPD ต้องเน้นข้อมูลอะไร? ER ต้องเน้นข้อมูลอะไร? เพราะอะไร?

B) ออกแบบ “กติกาการเรียกคนถัดไป” (Decision rules) เขียนกติกาเป็นข้อ ๆ ให้ชัดเจนว่า

- OPD: ตัดสินใจอย่างไรว่า “คนถัดไป” คือใคร?
- ER: ตัดสินใจอย่างไรว่า “คนถัดไป” คือใคร?
- กรณีคะแนน/ความเร่งด่วนเท่ากัน จะตัดสินด้วยอะไร? (tie-break)

C) ระบุ “กระบวนการที่ระบบต้องรองรับ” (Processes needed) ลองมองเป็นงานที่เกิดขึ้นจริงในหน่วยงาน แล้วเติมว่าแต่ละสถานการณ์จำเป็นหรือไม่

กระบวนการ	OPD	ER	หมายเหตุ
1) ลงทะเบียนผู้ป่วยใหม่			
2) ประเมินอาการเบื้องต้น/คัดแยก			
3) เรียก “คนถัดไป” ไปพบแพทย์			
4) ปรับข้อมูลเมื่ออาการเปลี่ยน (เช่น หนักขึ้น)			
5) ยกเลิกคิว/No-show (ผู้ป่วยไม่มา/กลับบ้าน)			
6) สรุปรายงาน (รอนานสุด/เฉลี่ย/จำนวนคงค้าง)			

D) เหตุการณ์จำลอง (Simulation) — คิดแบบระบบ สมมติว่ามีผู้ป่วยเข้ามาตามลำดับ: A, B, C, D (เวลาเพิ่มทีละ 2 นาที)

- ใน OPD: ถ้าผู้ป่วย C “ออกไปเข้าห้องน้ำ” แล้วกลับมา ระบบควรทำอะไร?
- ใน ER: หลังจากรอ 5 นาที ผู้ป่วย B มีอาการแย่ลง (เร่งด่วนขึ้น) ระบบควรทำอะไร?

เขียน “สิ่งที่ต้องเปลี่ยนในข้อมูล” และ “ขั้นตอนที่ต้องทำ” ของแต่ละกรณี

E) สรุป

1. ถ้าจำนวนผู้ป่วยเพิ่มจาก 10 เป็น 200 คน “ส่วนไหน” ของงานจะเริ่มวุ่น/ช้า/เสี่ยงผิดพลาดที่สุด?
2. ข้อมูลหรือขั้นตอนใดที่ถ้าขาดไป จะทำให้ระบบ “ไม่ยุติธรรม” หรือ “ไม่ปลอดภัย”?

แบบฝึกหัดที่ 3 (ในคาบ): พื้นฐาน algorithm: Trace การทำงานของ algorithm แบบง่าย ๆ

โจทย์: ค้นหาว่ามีชื่อ ``สมชัย" อยู่ในรายชื่อหรือไม่ด้วยวิธีการดังนี้



Algorithm 1 ค้นหาชื่อในรายชื่อ (เขียนแบบภาษามนุษย์)

Require: รายชื่อ L (ลำดับไม่แน่นอน), ชื่อที่ต้องการหา name

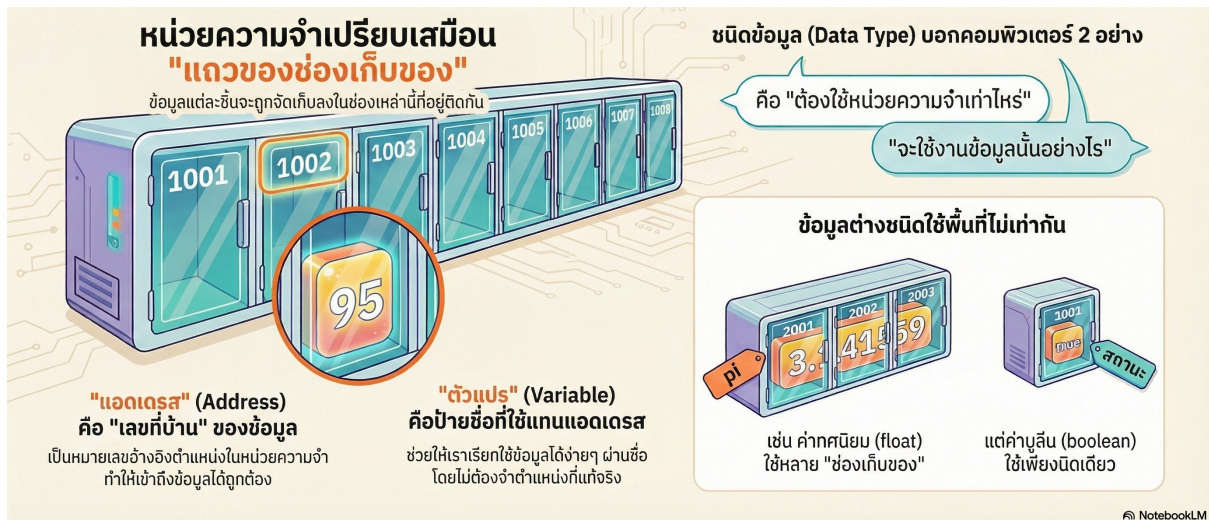
- 1: ในแต่ละรายชื่อ current ทำดังนี้: ▷ หยิบทีละชื่อจาก L เรียงลำดับ
 - 2: ดูว่า current ตรงกับ name หรือไม่
 - 3: ถ้าตรง **return true** ▷ พบชื่อที่ต้องการ
 - 4: ถ้าไม่ตรง วนกลับไปทำข้อ 2 จนกว่าจะหมดรายชื่อ แต่ถ้าหมดแล้ว **return false** ▷ ไม่พบชื่อที่ต้องการ
-

ให้ trace algorithm ด้านบนกับข้อมูลต่อไปนี้ แล้วตอบว่า True/False และต้องตรวจสอบกี่ครั้ง

รายชื่อ L	ชื่อที่ต้องการหา name	ผลลัพธ์ (True/False)	จำนวนครั้งที่ตรวจสอบ
[สมชาย, สมศรี, สมหมาย, สมปอง]	สมหมาย		
[สมชาย, สมศรี, สมหมาย, สมปอง]	สมชัย		
[สมชัย, สมชาย, สมศรี, สมหมาย]	สมหมาย		

3 Data in Memory (แบบเข้าใจจากภาพ)

3.1 Variables



ให้มอง *memory* เป็น "ตารางช่อง (cells)" แต่ละช่องเก็บค่าได้ และมีตำแหน่ง (*address*) กำกับอยู่

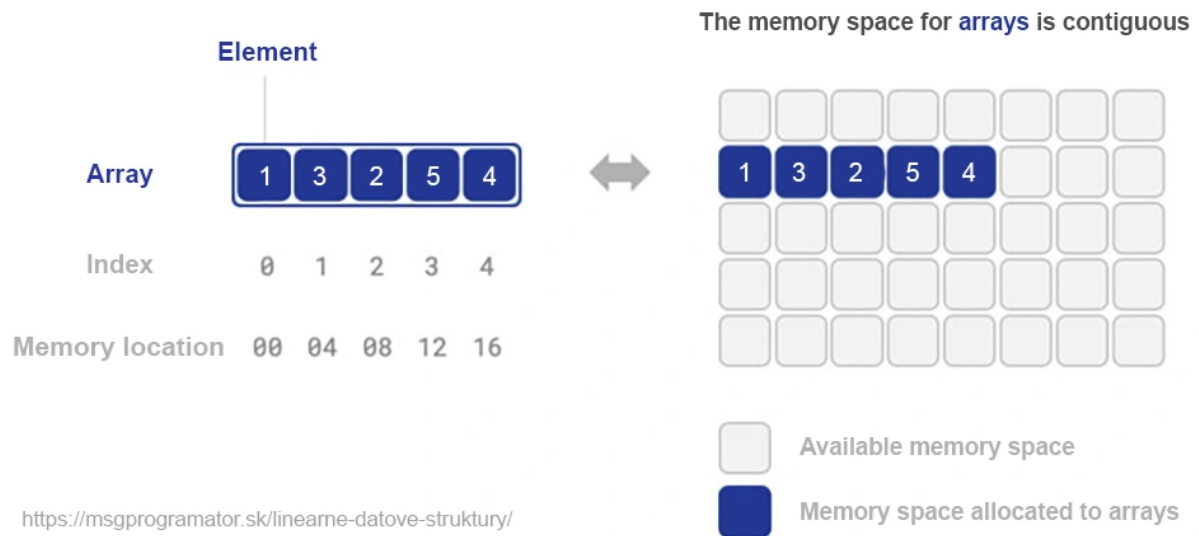
Actual Store	10001001	00010010	01100001	11000011	11110101	01001000	01000000
Address	777	778	779	780	781	782	783
Value	4745		'a'	3.14			
Data Type	short		char	float			
Variable Name	s		x	y			

แบบฝึกหัดที่ 4: อ่านภาพ *memory*

จากตาราง *memory* ที่เก็บค่าของตัวแปรด้านบนหมายความว่าอย่างไร:

คำถามชวนคิด: ถ้าเราต้องเก็บ "รายการข้อมูลหลายตัว" เราจะเก็บอย่างไรให้หยิบมาใช้ได้สะดวก?

3.2 Arrays



ลองพิจารณาว่าเพราะเหตุใดจึงจำเป็นต้อง

1. เรียงติดกัน
2. เก็บ data type ได้ประเภทเดียวกันเท่านั้น

4 ADT (Abstract Data Type): แยก “ทำอะไรได้” ออกจาก “ทำอย่างไร”

ADT คือ “สัญญา (interface)” ระบุ *operation* ที่ทำได้กับข้อมูลชนิดนั้น โดยไม่ผูกว่าต้องเก็บจริง ๆ แบบไหน

ตัวอย่าง: เราอาจนิยาม *List ADT* ได้ แม้ข้างในจะใช้ *array* หรือ *linked list* ก็ได้ (*implementation* ต่างกัน แต่ *operation* เหมือนกัน)

4.1 ตัวอย่าง ADT ที่จะได้รู้จัก

ADT	operation (ตัวอย่าง)	อินพุต/เอาต์พุต	หมายเหตุ
List ADT	<i>size()</i> , <i>isEmpty()</i> , <i>get(i)</i> , <i>insert(i,x)</i> , <i>remove(i)</i> , <i>find(x)</i>	i, x value/position	→ ใช้เก็บ “ลำดับ” ของข้อมูล
Stack ADT	<i>push(x)</i> , <i>pop()</i> , <i>top()</i> , <i>isEmpty()</i>	$x \rightarrow (-)$ / value	LIFO: last-in first-out
Queue ADT	<i>enqueue(x)</i> , <i>dequeue()</i> , <i>front()</i> , <i>isEmpty()</i>	$x \rightarrow (-)$ / value	FIFO: first-in first-out
Set ADT	<i>add(x)</i> , <i>remove(x)</i> , <i>contains(x)</i>	$x \rightarrow \text{true/false}$	ไม่สนใจลำดับ, ไม่ซ้ำ

ADT ต่างจาก data structure ตรงไหน

แบบฝึกหัดที่ 3: จำลอง operation ของ ADT (ไม่ต้องเขียนโค้ด)

A) List ADT

เริ่มต้น: $L = [10, 20, 30]$

ทำตามลำดับคำสั่ง แล้วเขียนผลลัพธ์ L หลังทำแต่ละขั้น

ข้อ	คำสั่ง	L หลังทำคำสั่ง	สิ่งที่ได้คืนออกมา
1	<i>insert(1, 15)</i> (แทรก 15 ที่ตำแหน่ง <i>index</i> 1)		
2	<i>remove(2)</i>		
3	<i>insert(0, 5)</i>		
4	<i>find(30)</i>		
5	<i>remove(3)</i>		

B) Stack ADT (LIFO)

เริ่มต้น: $S = []$

ทำคำสั่ง: *push(10)*, *push(20)*, *pop()*, *push(30)*, *top()*

เขียนผลลัพธ์สุดท้ายของ S และค่าที่ได้จาก *pop()/top()*

C) Queue ADT (FIFO)

เริ่มต้น: $Q = []$

ทำคำสั่ง: $enqueue(A)$, $enqueue(B)$, $dequeue()$, $enqueue(C)$, $front()$

เขียนผลลัพธ์สุดท้ายของ Q และค่าที่ได้จาก $dequeue()/front()$

5 Mini-case (อภิปรายในคาบ): เลือกโครงสร้างข้อมูลให้เหมาะกับปัญหา

สถานการณ์: คุณดูแล “ระบบลงทะเบียนกิจกรรม” ต้องรองรับงานต่อไปนี้:

- เพิ่มรายชื่อผู้สมัครใหม่ (*insert*)
- ยกเลิกการสมัคร (*delete*)
- ค้นหาชื่อ X สมัครหรือยัง (*search*)
- แสดงรายชื่อทั้งหมดตามลำดับเวลาสมัคร (*traversal*)

โจทย์: ให้ช่วยกันออกแบบว่า ADT ที่จะสร้างควรมีการดำเนินการอะไรบ้าง อีกทั้งในแต่ละการดำเนินการควรรับอะไรเป็น input และคืนอะไรเป็น output (ถ้ามี)

6 การบ้านหลังเรียน (Homework)

กำหนดส่ง: _____ | ส่งผ่าน: Microsoft Teams Assignment

ข้อ 1: จับคู่ “ปัญหา → ADT ที่เหมาะสม” (5 ข้อ)

ให้เลือก ADT ที่เหมาะสมที่สุด (เช่น *List ADT* / *Stack ADT* / *Queue ADT* / *Set ADT*) และให้เหตุผลสั้น ๆ

ข้อ	ปัญหา	ADT ที่เลือก	เหตุผล (สั้น ๆ)
1	ทำ <i>Undo/Redo</i> ในโปรแกรมเอกสาร (ย้อนกลับการกระทำล่าสุด)		
2	ระบบเรียกคิวบริการลูกค้า (มาก่อนบริการก่อน)		
3	เก็บรายการ “ของที่ต้องทำ” เรียงตามลำดับความสำคัญ (priority สูงก่อน) *ยังไม่ต้องรู้ heap*		
4	ตรวจว่ามีรหัสนักศึกษาซ้ำหรือไม่ (ต้องการเช็คสมาชิกอย่างรวดเร็ว)		
5	เก็บคะแนนสอบของนักศึกษาและต้องการเข้าถึงคะแนนตาม “ลำดับที่นั่งสอบ”		

ข้อ 2: ออกแบบ algorithm เบื้องต้นจาก ADT ที่กำหนดให้

คำชี้แจง

- อนุญาตให้เขียนเป็น *pseudocode* อธิบายด้วยคำพูด และ/หรือวาดภาพสถานะของโครงสร้างข้อมูล
- ห้าม อ้างถึง implementation (เช่น array/linked list) ให้คิดเฉพาะระดับ ADT และ operation ที่อนุญาตเท่านั้น

ส่วนที่ 1: ADT ที่กำหนดให้

WaitingLineADT<T> โครงสร้างนี้แทน “แถวรอ” แบบ **FIFO** (มาก่อน ได้ก่อน)

Operation ที่ใช้ได้

1. **Create()** → สร้างแถวว่าง
2. **Enqueue(L, x)** → เพิ่ม x เข้า ท้ายแถว
3. **Front(L)** → คืนค่า “คนหน้าสุด” โดย ไม่ลบออก
4. **Dequeue(L)** → ลบและคืนค่า “คนหน้าสุด”
5. **IsEmpty(L)** → คืนค่า True/False
6. **Size(L)** → คืนค่าจำนวนสมาชิกในแถว

กติกาเมื่อแถวว่าง ถ้าเรียก **Front(L)** หรือ **Dequeue(L)** ในกรณีที่ **IsEmpty(L) = True** ให้ถือว่าผลลัพธ์เป็น **ERROR** และสถานะของแถว ไม่เปลี่ยน

ส่วนที่ 2: โจทย์สถานการณ์จริง

สถานการณ์: ระบบเรียกคิวบริการ “เคาน์เตอร์เดียว” ศูนย์บริการมี 1 เคาน์เตอร์ ทำงานตามเหตุการณ์ที่เข้ามาเป็นลำดับ

งานที่ต้องทำ ให้นักศึกษาอธิบายขั้นตอนวิธี (algorithm) ที่จะใช้ WaitingLineADT แก้ปัญหานี้อย่างไร โดยต้องมี:

1. อธิบายว่า ``จะเก็บข้อมูลอะไรไว้ในแถว"
2. เขียนเป็น *pseudocode* สั้น ๆ หรือเขียนเป็นลำดับขั้นก็ได้ แต่ต้องชัดเจนว่าเรียก operation ใดบ้าง

สถานการณ์: ร้านแห่งหนึ่งเปิดให้รอคิวได้ก่อนร้านเปิด 1 ชั่วโมง หลังจากร้านเปิดให้จองคิว มะลิ และ นนท์ เข้ามาเป็นกลุ่มแรก และเข้าแถวโดยนนท์แสดงถึงความเป็นสุภาพบุรุษ จึงให้มะลิเข้าคิวก่อน ต่อมา เจ้าของร้านเดินมาเช็คคิวว่าซื้ออะไรอยู่หน้าสุด แล้วก็เรียกคนในคิวเข้าร้านไป 1 คน ระหว่างนั้น พลอยก็เข้ามาจะใช้บริการ จึงได้ทำการเข้าคิว แต่เนื่องจากตอนที่กำลังบริการมะลียุ่้นั้น เจ้าของร้านไม่ทราบว่ามีคนมาเข้าคิวเพิ่มใหม่ จึงได้ตรวจสอบจำนวนคนในคิว และได้เรียกคนเข้าไปในร้าน 2 คน แล้วก็ตรวจสอบว่าแถวว่างหรือยัง

ให้เริ่มจาก $L = \text{Create}()$ และใช้สถานการณ์ที่อธิบายไว้ด้านบนมาเขียนลำดับคำสั่งของ ADT ที่ต้องเรียกใช้งาน ให้นักศึกษารอการถาม: ``ผลลัพธ์ที่คืนค่า" และ ``สถานะแถวหลังจบคำสั่ง" (เขียนสถานะแถวเป็น [front ... back])

Step	สถานการณ์ (ภาษาพูด)	Operation	Return value	State of L (front→back)
1	ร้านเปิดรอคิว	Create()	L	[]
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				

*** ครึ่งหน้าเราจะมาลองใช้ Python เบื้องต้น โดยที่อาจารย์จะเตรียมโค้ดการทำงานไว้ให้นักศึกษาแค่เรียกใช้งาน เพื่อให้ นักศึกษาค้นเคยกับการใช้ Python ใน vscode บ้าง แต่การเรียกใช้งานในโค้ด จะแตกต่างจากที่เขียนในการบ้านนิดหน่อย เช่น $\text{Front}(L)$ ในโค้ดจะเขียนเป็น $L.\text{front}()$

หมายเหตุ ต้องยอมรับว่าสอนยากเพราะจริง ๆ แล้ว ในมหาวิทยาลัยอื่น ๆ จะสอนวิชานี้หลังจากที่นักศึกษาได้เรียนวิชาการเขียนโปรแกรมเบื้องต้นในปี 1 เทอม 1 และวิชาการออกแบบเชิงวัตถุ (OOP) ในปี 1 เทอม 2 แล้วเรียนวิชาโครงสร้างข้อมูลในปี 2 เทอม 1 แต่เนื่องจากพวกเราไม่เคยเขียนโปรแกรมมาก่อน ทำให้ต้องสอนแบบเน้นเขียนมือออกแบบขั้นตอน แล้วเรียกโค้ดง่าย ๆ ในสัปดาห์ถัดไป ไม่ลงลึก implementation ด้วยตัวเอง (แต่หลังจากที่เรียนวิชา programming ผ่านแล้วขอแนะนำให้นักศึกษาลองเขียน implementation ด้วยตัวเองอีกรอบ)